Oracle DBA

Advanced Interview Preparation Guide

Senior / Advanced Level — Deep Dive Topics

- Oracle Multitenant (CDB / PDB)
- ASM (Automatic Storage Management)
- Data Guard — Advanced & Deep Dive
- RAC — Real Application Clusters
- GoldenGate — Replication & Migration
- Advanced Performance Tuning
- Partitioning — Tables & Indexes
- Database Upgrade & Migration
- Security — TDE, VPD & Auditing
- Troubleshooting — Advanced Scenarios

Simple Language | Topic Summary + Description + Example | Multi-Step Answers

April 2026 | Oracle Database 19c / 21c

SECTION 1: Oracle Multitenant Architecture (CDB / PDB)

📌 **TOPIC SUMMARY**

Oracle Multitenant is the modern database architecture introduced in Oracle 12c. Instead of having 10 separate databases on a server, you create ONE Container Database (CDB) that holds many Pluggable Databases (PDBs) inside — sharing resources but staying fully isolated.

📖 **DESCRIPTION**

A CDB (Container Database) is the parent database that contains the Oracle system files, SGA, and background processes. PDBs (Pluggable Databases) are self-contained databases inside the CDB — each with its own schema, users, tablespaces, and data. PDBs share the CDB's memory and background processes, making it more resource-efficient. You can easily unplug a PDB from one CDB and plug it into another — making migrations, cloning, and upgrades much faster. Every new Oracle installation defaults to CDB architecture. Non-CDB is deprecated from Oracle 21c onwards.

💡 **REAL-LIFE EXAMPLE**

Think of a CDB as a large apartment building. The building's foundations, electricity, water supply, and security (background processes, SGA) are shared by all apartments. Each apartment (PDB) is completely private — different owner, different furniture, different door key. You can move an apartment from one building to another (unplug/plug PDB) without disturbing the neighbours.

1.1 CDB / PDB Core Q&A

Q: What is the difference between CDB and PDB? When should you use Multitenant?

A:

CDB (Container Database):

- The root container — holds Oracle's data dictionary (CDB$ROOT), seed PDB (PDB$SEED), and all user PDBs.
- Has one SGA, one set of background processes — shared by all PDBs.
- You log in to CDB as SYS or SYSTEM with 'AS SYSDBA'.

PDB (Pluggable Database):

- A fully self-contained database inside the CDB — own users, tablespaces, schemas.
- Applications connect directly to the PDB, not to the CDB root.
- Can be opened, closed, plugged, unplugged, and cloned independently.

When to use:

- Use Multitenant (CDB+PDB) for all new Oracle 19c/21c databases — it is the standard and Non-CDB is deprecated.
- Each application gets its own PDB — full isolation with shared resource efficiency.

Q: How do you create a new PDB inside an existing CDB?

A:

- Step 1 — Connect to CDB root as SYSDBA: sqlplus / as sysdba
- Step 2 — Create a PDB from the seed (PDB$SEED):

```
CREATE PLUGGABLE DATABASE hrpdb
 ADMIN USER pdbadmin IDENTIFIED BY PdbPass123!
 DEFAULT TABLESPACE users
 DATAFILE '/u02/oradata/ORCL/hrpdb/users01.dbf' SIZE 500M AUTOEXTEND ON
 FILE_NAME_CONVERT = ('/u02/oradata/ORCL/pdbseed/','/u02/oradata/ORCL/hrpdb/');
```

- Step 3 — Open the new PDB: ALTER PLUGGABLE DATABASE hrpdb OPEN;
- Step 4 — Save state so it opens automatically after CDB restart: ALTER PLUGGABLE DATABASE hrpdb SAVE STATE;
- Step 5 — Verify: SELECT con_id, name, open_mode FROM v\$pdbs;

Q: What is PDB$SEED and can you modify it?

A:

PDB$SEED is the template PDB that Oracle uses when you create new PDBs using CREATE PLUGGABLE DATABASE.

It is like a master copy — every new PDB is initially a clone of PDB$SEED.

Can you modify it?

- By default: NO. PDB$SEED is read-only.
- To modify (e.g., to pre-install tablespaces in all future PDBs):

```
ALTER PLUGGABLE DATABASE pdb\$seed OPEN READ WRITE FORCE;
-- Make your changes --
ALTER PLUGGABLE DATABASE pdb\$seed OPEN READ ONLY FORCE;
```

Best practice: Do not modify PDB$SEED unless you have a strong reason. It affects every future PDB created from it.

Q: How do you clone a PDB? What are the different PDB clone methods?

A:

Method 1 — Clone from an existing PDB (local clone):

- Close the source PDB in read-only mode: ALTER PLUGGABLE DATABASE hrpdb CLOSE; ALTER PLUGGABLE DATABASE hrpdb OPEN READ ONLY;
- Create the clone:

```
CREATE PLUGGABLE DATABASE hrpdb_dev FROM hrpdb
 FILE_NAME_CONVERT = ('/hrpdb/','/hrpdb_dev/');
```

- Open the clone: ALTER PLUGGABLE DATABASE hrpdb_dev OPEN;

Method 2 — Hot clone (no need to close source, requires local undo):

```
CREATE PLUGGABLE DATABASE hrpdb_test FROM hrpdb@dblink_src NO DATA;
```

Method 3 — Snapshot Clone (very fast, space-efficient, requires ACFS storage):

```
CREATE PLUGGABLE DATABASE hrpdb_snap FROM hrpdb SNAPSHOT COPY;
```

Snapshot Clone is the fastest — creates a PDB in seconds because it shares blocks with the source. Best for dev/test environments.

⭐ Q: What happens when you UNPLUG a PDB? How do you plug it into another CDB?

A:

UNPLUG — Export the PDB to an XML manifest file:

- Close the PDB first: `ALTER PLUGGABLE DATABASE hrpdb CLOSE IMMEDIATE;`
- Unplug it:

```
ALTER PLUGGABLE DATABASE hrpdb
  UNPLUG INTO '/backup/hrpdb.xml';
```

- Drop from the source CDB: `DROP PLUGGABLE DATABASE hrpdb KEEP DATAFILES;`

PLUG — Import into a different CDB:

- On the target CDB, check compatibility first:

```
SET SERVEROUTPUT ON
BEGIN
  IF DBMS_PDB.CHECK_PLUG_COMPATIBILITY(pdb_descr_file=>'/backup/hrpdb.xml')
THEN
    DBMS_OUTPUT.PUT_LINE('Compatible!');
  END IF;
END; /
```

- Create the PDB from the XML:

```
CREATE PLUGGABLE DATABASE hrpdb USING '/backup/hrpdb.xml'
  COPY FILE_NAME_CONVERT = ('/old/path/','/new/path/');
```

- Open it: `ALTER PLUGGABLE DATABASE hrpdb OPEN;`

Q: Scenario: You need to apply a patch to one PDB without affecting other PDBs in the same CDB. Is that possible?

A:

This is one of the BIG advantages of Multitenant architecture.

For database-level changes (SQL changes via datapatch):

- Since Oracle 12.2, datapatch can apply patches at the PDB level individually.
- You can patch one PDB without touching others in the same CDB.

For Oracle software patches (OPatch — binary patches):

- These apply to the entire Oracle Home (shared by all PDBs) — cannot be isolated per PDB.

Common scenario — upgrading a PDB to a newer version:

- ▶ Unplug the PDB from the older CDB.
- ▶ Plug it into a CDB running the newer Oracle version.
- ▶ Run datapatch on the new CDB for just that PDB.

This is called a 'rolling PDB upgrade' — zero downtime on other PDBs.

✅ Key Points to Remember

✔ CDB = apartment building (shared infrastructure). PDB = individual apartment (isolated data).
✔ PDB$SEED = master template for new PDBs — read-only by default.
✔ Save state: ALTER PLUGGABLE DATABASE name SAVE STATE — auto-open after CDB restart.
✔ Unplug = export PDB to XML. Plug = import XML into another CDB.
✔ Key views: V$PDBS, CDB_USERS, CDB_TABLESPACES, DBA_PDBS
✔ Connect to PDB: ALTER SESSION SET CONTAINER = pdbname OR use TNS service name.

SECTION 2: ASM — Automatic Storage Management

📌 **TOPIC SUMMARY**

ASM (Automatic Storage Management) is Oracle's built-in volume manager and file system specifically designed for Oracle database files. It automatically manages disk space distribution, mirroring, and rebalancing — so the DBA does not need to manually manage disk partitions or file systems.

📖 **DESCRIPTION**

Instead of putting Oracle files on a normal file system (like ext4 or XFS), you give raw disks to ASM. ASM creates 'disk groups' from these disks and stores Oracle files (data files, redo logs, control files, archive logs, RMAN backups) in these disk groups. ASM automatically stripes data across all disks for performance (no hot disks) and mirrors data for redundancy (no single point of failure). When you add a new disk, ASM automatically 'rebalances' by moving data around so all disks are evenly used — no manual work needed. ASM runs as a separate Oracle instance (the +ASM instance) alongside the database instance.

💡 **REAL-LIFE EXAMPLE**

Think of ASM like a RAID system that Oracle manages itself. Instead of your IT team manually setting up RAID-10 across 8 disks and creating LUNs and mount points, you just say 'ASM, here are 8 disks — create a NORMAL redundancy disk group'. ASM handles everything: striping for speed, mirroring for safety, and rebalancing when you add new disks. No storage admin needed for the Oracle DBA's storage.

2.1 ASM Core Q&A

Q: What is an ASM Disk Group? What are the redundancy levels?

A:

An ASM Disk Group is a logical pool of physical disks that ASM manages as one unit. All Oracle database files are stored in disk groups, not on file systems.

Redundancy levels (3 types):

- EXTERNAL REDUNDANCY: No mirroring by ASM. Relies on hardware RAID. Use when the storage hardware already provides redundancy. Least overhead.
- NORMAL REDUNDANCY: 2-way mirroring (each extent has 2 copies on different disks). Requires minimum 2 failure groups (disks on different controllers). Most common in production.
- HIGH REDUNDANCY: 3-way mirroring (3 copies of every extent). Maximum protection. Requires minimum 3 failure groups. Uses more disk space.

Create a disk group example:

```
CREATE DISKGROUP DATA NORMAL REDUNDANCY
  FAILGROUP fg1 DISK '/dev/sdb' NAME disk1
  FAILGROUP fg2 DISK '/dev/sdc' NAME disk2
  ATTRIBUTE 'compatible.asm'='19.0','compatible.rdbms'='19.0';
```

⭐ Q: What is a Failure Group in ASM? Why is it important?

A:

A Failure Group is a set of ASM disks that share a common point of failure — usually disks connected to the same controller, same shelf, or same power supply.

ASM mirrors data across failure groups — NOT just across individual disks. This ensures that even if an entire controller or shelf fails, your data is safe.

Example:

- Server has 2 disk controllers — CTRL-A (disks 1,2,3) and CTRL-B (disks 4,5,6).
- Failure Group 1 = disks 1,2,3 on CTRL-A. Failure Group 2 = disks 4,5,6 on CTRL-B.
- ASM mirrors every extent: one copy on CTRL-A, one copy on CTRL-B.
- If CTRL-A dies completely, all data is still available on CTRL-B.

If you put all disks in ONE failure group and use NORMAL REDUNDANCY, you get 2 copies on the SAME physical disks — worthless for protection.

Q: What is ASM Rebalancing? When does it happen?

A:

Rebalancing is the automatic process ASM performs to evenly distribute data across all disks in a disk group.

When rebalancing happens:

- ▶ When you ADD a new disk to a disk group — ASM moves data from existing disks to the new disk until all are equally used.
- ▶ When you DROP a disk — ASM moves all data off that disk before dropping it.
- ▶ When you resize a disk.

Control rebalancing speed with ASM_POWER_LIMIT:

```
ALTER DISKGROUP data REBALANCE POWER 8;
-- Range: 0 (suspend) to 1024 (Oracle 12.2+). Default = 1 (slowest, least impact).
-- Higher value = faster rebalance, but uses more I/O.
```

Monitor rebalancing progress:

```
SELECT * FROM v\$asm_operation;
-- Shows estimated time remaining and progress.
```

Q: What is the difference between ASM metadata views: VASM_DISK, VASM_DISKGROUP, V$ASM_FILE?

A:

V$ASM_DISK: Shows all physical disks known to ASM — their path, size, free space, disk group membership, status (NORMAL, MISSING, DROPPING).

V$ASM_DISKGROUP: Shows all disk groups — name, type (NORMAL/HIGH/EXTERNAL), total size, free space, state (MOUNTED/DISMOUNTED).

V$ASM_FILE: Shows all files stored in ASM — their reference number, file type (DATAFILE, ARCHIVELOG, BACKUPSET), size.

V$ASM_ALIAS: Shows the full path of ASM files (e.g., +DATA/ORCL/DATAFILE/users.345.1000123456).

V$ASM_CLIENT: Shows which database instances are currently using which disk groups.

Q: Scenario: One disk in an ASM disk group shows status 'OFFLINE'. What do you do?

A:

Step 1 — Check the disk status:

```
SELECT name, path, state, mode_status, total_mb, free_mb FROM v\$asm_disk
WHERE group_number > 0 ORDER BY group_number, disk_number;
```

Step 2 — Check if it is a temporary I/O issue (try to bring it back online):

```
ALTER DISKGROUP data ONLINE DISK disk1;
```

Step 3 — If disk is physically failed, check ASM has been rebalancing to protect data:

```
SELECT * FROM v\$asm_operation;  -- Check if rebalance is running
```

Step 4 — If disk is dead permanently, drop it:

```
ALTER DISKGROUP data DROP DISK disk1;
-- ASM automatically moves all data off this disk before dropping (rebalance).
```

Step 5 — Add a replacement disk:

```
ALTER DISKGROUP data ADD FAILGROUP fg1 DISK '/dev/sdb' NAME disk1_new;
```

Step 6 — Monitor the rebalance:

```
SELECT operation, est_minutes FROM v\$asm_operation;
```

Q: What is the ASM ASMCMD utility? Name 5 useful commands.

A:

ASMCMD is a command-line tool (similar to Linux shell commands) for managing ASM disk groups and files. Run it as: asmcmd (as oracle user with ORACLE_SID=+ASM).

lsdg: List all disk groups and their usage (like 'df' in Linux). Example: lsdg

ls: List files in a disk group directory. Example: ls +DATA/ORCL/DATAFILE/

mkdir: Create a new directory in a disk group. Example: mkdir +DATA/newdb

cp: Copy files between ASM and filesystem. Example: cp +DATA/ORCL/spfileORCL.ora /backup/

md_backup: Backup ASM disk group metadata (critical for disaster recovery). Example: md_backup -b /backup/asm_meta.bkp

lsod: List open database files (shows which DB instance has files open).

✅ Key Points to Remember

✔ ASM = Oracle's own storage manager. Replaces file systems (ext4, XFS) for Oracle files.
✔ Disk Group = logical pool of disks. Redundancy: EXTERNAL (none), NORMAL (2-way), HIGH (3-way).
✔ Failure Group = disks that share a hardware failure point — always spread failure groups across controllers.
✔ Rebalancing = automatic redistribution when disks are added/removed. Control speed with ASM_POWER_LIMIT.
✔ Key tool: asmcmd. Key views: VASM_DISK, VASM_DISKGROUP, VASM_FILE, VASM_OPERATION.

SECTION 3: Data Guard — Advanced Deep Dive

📌 **TOPIC SUMMARY**

Advanced Data Guard topics include the Data Guard Broker (DGMGRL), Fast-Start Failover (FSFO), transport modes (SYNC vs ASYNC), protection modes, standby redo logs, MRP process, gap resolution, and Data Guard with RAC.

📖 **DESCRIPTION**

At an advanced level, DBAs must understand the complete Data Guard architecture including how redo transport works (LNS process sending redo, RFS process receiving on standby), when to use synchronous vs asynchronous transport, how the MRP (Managed Recovery Process) applies redo on the standby, and how to manage Data Guard using the Broker (DGMGRL) tool which simplifies switchover/failover to single commands. Fast-Start Failover (FSFO) automates failover without human intervention — critical for environments requiring RPO near zero and RTO under a minute.

💡 **REAL-LIFE EXAMPLE**

Without Data Guard Broker, doing a switchover requires running 6–8 SQL commands on both primary and standby in the correct sequence. With DGMGRL: you type 'SWITCHOVER TO standby_name' and it handles everything automatically — validates readiness, executes the role change, updates the configuration. It is like the difference between flying a plane manually versus using autopilot.

3.1 Data Guard Broker (DGMGRL) Q&A

Q: What is Data Guard Broker and why should you use it instead of SQL commands?

A:

Data Guard Broker is a framework that automates and simplifies the management of an Oracle Data Guard configuration. It consists of the Broker process (DMON) running on each database and the DGMGRL command-line tool.

Advantages over manual SQL commands:

- Switchover in ONE command: DGMGRL> SWITCHOVER TO standby_db; (vs 8 manual SQL steps that must be done in exact order)
- Failover in ONE command: DGMGRL> FAILOVER TO standby_db;
- Health monitoring — Broker continuously monitors all members and reports problems.
- Fast-Start Failover — Broker enables AUTOMATIC failover with the Observer.
- Consistent configuration — all settings stored in the Broker configuration file, not scattered across init parameters.

Set up Broker (enable on both primary and standby):

```
ALTER SYSTEM SET dg_broker_start=TRUE SCOPE=BOTH;
```

Create Broker configuration:

```
dgmgrl /
```

```
CREATE CONFIGURATION dg_config AS PRIMARY DATABASE IS orcl CONNECT IDENTIFIER IS orcl;
ADD DATABASE orclstby AS CONNECT IDENTIFIER IS orclstby MAINTAINED AS PHYSICAL;
ENABLE CONFIGURATION;
```

⭐ Q: What is the difference between SYNC and ASYNC redo transport in Data Guard?

A:

SYNC (Synchronous): Primary waits for confirmation from standby that redo has been received before committing. Zero data loss guaranteed. Used in MAXIMUM AVAILABILITY and MAXIMUM PROTECTION modes. Slight performance overhead on primary if network is slow.

ASYNC (Asynchronous): Primary does NOT wait for standby confirmation before committing. Higher primary performance. Small risk of data loss (seconds or minutes) on failover. Used in MAXIMUM PERFORMANCE mode. Best for long-distance DR where network latency is high.

Which to use?

- SYNC: Same data center or fast low-latency network (< 5ms RTT). Banking, financial, zero-tolerance systems.
- ASYNC: Cross-city or cross-country DR (high latency). Systems where small data loss is acceptable in a disaster.

Configure transport mode in Broker: `DGMGRL> EDIT DATABASE orclstby SET PROPERTY LogXptMode='ASYNC';`

Q: What is Fast-Start Failover (FSFO)? How does it work?

A:

FSFO (Fast-Start Failover) is a Data Guard feature that allows AUTOMATIC failover to the standby — without any DBA intervention — when the primary database fails.

How FSFO works:

- ▶ An Observer process (runs on a third server — NOT on primary or standby) continuously monitors the primary.
- ▶ If the primary becomes unavailable AND the standby is in sync (or within the configured lag threshold), the Observer triggers automatic failover.
- ▶ The standby is promoted to primary automatically.
- ▶ Application connections are redirected to the new primary via SCAN listener or TNS.

Enable FSFO in Broker:

```
DGMGRL> ENABLE FAST_START FAILOVER;
DGMGRL> START OBSERVER;  -- Run this on the Observer server
```

Key FSFO parameters:

- FastStartFailoverThreshold — seconds of primary unavailability before failover triggers (default 30 sec).
- FastStartFailoverLagLimit — max accepted data lag on standby for FSFO to activate (used with ASYNC transport).

Q: What is a Redo Gap in Data Guard? How do you detect and resolve it?

A:

A Redo Gap (also called Archive Gap) occurs when the standby is missing some archive log files that were created on the primary — usually because of a network outage between primary and standby.

Detect a gap:

```
-- On PRIMARY:
SELECT thread#, low_sequence#, high_sequence# FROM v\$archive_gap;
-- If rows returned: there is a gap on the standby.

-- On STANDBY:
SELECT thread#, sequence# FROM v\$archived_log WHERE applied='NO'
MINUS
SELECT thread#, sequence# FROM v\$archived_log WHERE applied='YES';
```

Automatic gap resolution:

- FAL (Fetch Archive Log) process automatically fetches missing logs from primary. This is automatic when FAL_SERVER is configured.

Manual gap resolution (if auto-fetch is not working):

- Copy the missing archive log files from primary to standby manually.
- Register them on standby: ALTER DATABASE REGISTER LOGFILE '/path/to/archivelog';
- Verify MRP resumes applying: SELECT process, status, sequence# FROM v\$managed_standby;

Q: What is Multi-Instance Redo Apply (MIRA) in Data Guard? (Oracle 12.2+ feature)

A:

Before Oracle 12.2, the MRP (Managed Recovery Process) could only run on ONE node of a RAC standby. This was a bottleneck — only one instance was applying redo.

MIRA (Multi-Instance Redo Apply) — introduced in Oracle 12.2 — allows the MRP to run on MULTIPLE nodes of a RAC standby simultaneously, applying redo in parallel.

Enable MIRA on RAC standby:

```
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
  DISCONNECT FROM SESSION INSTANCES ALL;
-- 'INSTANCES ALL' means all RAC nodes participate in redo apply.
```

Or specify a specific number of instances:

```
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
  DISCONNECT FROM SESSION INSTANCES 2;
```

Benefit: Much higher redo apply throughput for very busy primary databases with high redo generation.

✅ Key Points to Remember

✔ Broker (DGMGRL) = automated DG management. Set dg_broker_start=TRUE on both sides.
✔ SYNC = zero data loss (waits for standby ACK). ASYNC = better performance, tiny data loss risk.
✔ FSFO = automatic failover with Observer process. Observer must be on a third separate server.
✔ Redo Gap = missing archive logs on standby. Auto-resolved by FAL_SERVER parameter.
✔ MIRA (12.2+) = multi-node redo apply on RAC standby for higher apply throughput.
✔ Key views: V$DATAGUARD_STATUS, V$MANAGED_STANDBY, V$ARCHIVE_DEST_STATUS, V$ARCHIVE_GAP

SECTION 4: RAC — Real Application Clusters (Advanced)

📌 **TOPIC SUMMARY**

Advanced RAC topics include Grid Infrastructure, Cluster Ready Services (CRS), Virtual IP (VIP), SCAN (Single Client Access Name), Cache Fusion internals, interconnect troubleshooting, and RAC-specific performance tuning. These are heavily tested at the senior DBA level.

📖 **DESCRIPTION**

RAC runs on top of Oracle Grid Infrastructure (GI) which contains two main components: Clusterware (manages the cluster — node health, resource management, VIPs) and ASM (manages shared storage). Each node runs an Oracle database instance, and all instances share the same database files stored in ASM. The private interconnect is the high-speed network between nodes used exclusively for Cache Fusion (transferring data blocks between node memory) and cluster heartbeat. The public network carries application traffic. SCAN (Single Client Access Name) provides a single hostname for clients — even as nodes are added/removed, the client connection string never changes.

💡 **REAL-LIFE EXAMPLE**

Imagine a 3-node RAC cluster as a team of 3 workers sharing one filing cabinet (ASM). Each worker has their own desk (instance with SGA and processes). They pass files between desks via a private office intercom (interconnect) — this is Cache Fusion. Clients call ONE reception number (SCAN) and get connected to any available worker. If one worker calls in sick (node crash), the other two keep working and reception redirects calls automatically.

4.1 RAC Grid Infrastructure Q&A

Q: What is Oracle Grid Infrastructure? What are its two main components?

A:

Oracle Grid Infrastructure (GI) is the base software layer that must be installed BEFORE the Oracle database in a RAC environment. It provides the cluster management and shared storage capabilities.

Clusterware: Manages cluster resources — starts/stops instances, manages VIPs, monitors node health, handles failover when a node crashes. Key daemons: CSSD (cluster sync), CRSD (cluster resource), OHASD.

ASM (Automatic Storage Management): Manages shared disk storage for all Oracle database files in the cluster. Runs as the +ASM instance. All RAC nodes connect to the same ASM disk groups.

Key GI commands:

- crsctl check cluster -all — check cluster health on all nodes
- crsctl stat res -t — list all cluster resources and their status

- srvctl status database -db orcl — check database status on all nodes
- srvctl start database -db orcl — start DB on all nodes

⭐ Q: What is SCAN (Single Client Access Name)? Why is it better than VIP?

A:

VIP (Virtual IP): Each RAC node has its own VIP address. When a node crashes, its VIP moves to another node and sends TCP RESET to waiting clients — so they fail fast instead of waiting for TCP timeout. BUT: clients need to know ALL VIP addresses in their connection string.

SCAN (Single Client Access Name): A single DNS name that resolves to 3 SCAN IP addresses (Oracle recommends 3 for load balancing and redundancy). Clients only need ONE connection string regardless of how many nodes the cluster has.

Example connection string with SCAN:

```
jdbc:oracle:thin:@//db-scan.company.com:1521/hrpdb
-- ONE hostname. Works even if cluster grows from 3 to 10 nodes.
```

Without SCAN (old VIP method):

```
(ADDRESS=(HOST=node1-vip)(PORT=1521))(ADDRESS=(HOST=node2-vip)(PORT=1521
))
-- Must update all application configs when cluster changes.
```

SCAN is always the recommended connection method for Oracle RAC from 11g onwards.

Q: What is Cache Fusion? How does it work internally?

A:

Cache Fusion is the technology that allows RAC nodes to share data blocks directly through their memory (via the private interconnect) without having to write to disk and read back.

How it works — Example:

- Node 1 reads the 'EMPLOYEES' block from disk into its buffer cache.
- Node 2 needs the same 'EMPLOYEES' block to run an UPDATE.
- Instead of Node 2 reading from disk, it asks Node 1 via the interconnect.
- Node 1 sends the block directly to Node 2's memory over the private network.
- Node 2 modifies the block. Node 1 keeps a 'past image' in case of recovery.

GRD (Global Resource Directory):

Tracks which instance holds which block in memory. Each block has a 'master node' that records where every copy of that block lives.

Performance issue — Cluster Wait events:

- 'gc buffer busy acquire' — block is being transferred from another node.

- 'gc current block 2-way' — normal Cache Fusion block transfer (2 nodes involved).
- High gc waits = too much data sharing between nodes = consider partitioning or service separation.

Q: What is the Private Interconnect in RAC? What happens if it fails?

A:

The private interconnect is a high-speed, dedicated network (usually 10 GbE or InfiniBand) between RAC nodes used exclusively for:

- Cache Fusion — block transfers between node memory caches
- Cluster heartbeat — each node confirms it is alive every second
- Global Enqueue Service (GES) messaging — lock coordination

What happens if the interconnect fails?

- ▶ The cluster detects the failure through missed heartbeats.
- ▶ CSSD (Cluster Synchronization Services) uses a 'voting disk' to determine which node survives.
- ▶ The node that CANNOT see the majority of voting disks will perform a 'node eviction' (REBOOT itself).
- ▶ Surviving nodes take over the failed node's services.

This is called 'Split Brain' protection — prevents two nodes both thinking they are the primary, which would corrupt data.

Monitor interconnect health:

```
SELECT * FROM v\$cluster_interconnects;
oifcfg getif  -- shows configured interfaces
```

Q: Scenario: One RAC node is evicted from the cluster unexpectedly. What are your first steps?

A:

- ▶ Check the cluster alert log on ALL nodes:

```
$GRID_HOME/log/<hostname>/alert<hostname>.log
-- Look for: 'Eviction initiated' / 'CSS clears node' / 'CSSD starting node eviction'
```

- ▶ Check the CSSD trace file (most detailed cluster info):

```
$GRID_HOME/log/<hostname>/cssd/ocssd.trc
```

- ▶ Check if the eviction was due to missed heartbeats (interconnect issue):

```
grep 'Eviction' $GRID_HOME/log/<hostname>/cssd/ocssd.trc | tail -20
```

- ▶ Check network interface statistics on the evicted node (was it overloaded?):

```
netstat -i  OR  sar -n DEV 1 10
```

- ▶ Check if it was an OS issue (memory pressure, hung processes):

Look at /var/log/messages for 'kernel: OOM killer' or hung task warnings

▶ Once root cause is found, rejoin the node to the cluster:

```
crsctl start cluster  -- on the evicted node
```

▶ Verify all resources are back:

```
crsctl stat res -t
```

✅ Key Points to Remember

✔ Grid Infrastructure = Clusterware (cluster management) + ASM (shared storage). Install BEFORE DB.
✔ SCAN = one DNS name for all cluster nodes. Always use SCAN in application connection strings.
✔ Cache Fusion = block transfer between node memories via private interconnect (no disk I/O).
✔ GRD (Global Resource Directory) tracks which node holds each data block.
✔ Voting disk = tiebreaker during split-brain. Node that cannot see majority of voting disks = evicted.
✔ Key tools: crsctl, srvctl, asmcmd. Key views: GV$INSTANCE, V$CLUSTER_INTERCONNECTS

SECTION 5: Oracle GoldenGate — Replication & Migration

📌 **TOPIC SUMMARY**

Oracle GoldenGate is a real-time data replication and integration tool. Unlike Data Guard (which replicates everything), GoldenGate can replicate specific tables, transform data during replication, and replicate between completely different database platforms — Oracle to MySQL, Oracle to SQL Server, and more.

📖 **DESCRIPTION**

GoldenGate works by reading the database redo/archive logs (log-based capture) and extracting committed transactions. These changes are written to trail files (compressed binary files). A pump process sends trail files to the target server. A Replicat process reads the trail files and applies the changes to the target database using SQL. Key processes: Extract (reads source logs), Data Pump (sends trail files), Replicat (applies to target). GoldenGate supports bi-directional replication (both sides write), conflict detection, and table-level filtering — making it ideal for zero-downtime migrations and active-active configurations.

💡 **REAL-LIFE EXAMPLE**

Imagine you need to migrate a 10 TB Oracle 11g database to a new Oracle 19c server with ZERO downtime. With GoldenGate: you set it up to replicate all changes from 11g to 19c in real time. While replication runs, you do an initial load of historical data. Once 19c is caught up (lag = 0), you switch the application to the new server in minutes. The old server handled thousands of transactions during the migration — all captured and replicated. Without GoldenGate, this would require hours of downtime.

5.1 GoldenGate Architecture Q&A

Q: What are the main GoldenGate processes? Explain each.

A:

Manager: The 'mother process' of GoldenGate. Must be running before any other process. Manages all other processes, listens on a port for commands, monitors trails, and handles reporting. Start: GGSCI> START MGR

Extract: Reads committed transactions from the source database's redo/archive logs. Writes changes to local trail files. There are 2 types of Extract: (1) Primary Extract — reads online redo logs in real time. (2) Data Pump Extract — reads local trail files and forwards them to the target server.

Trail File: Binary files where extracted changes are stored temporarily. On source server = local trail (e.g., ./dirdat/et). On target server = remote trail (e.g., ./dirdat/rt). Compressed and encrypted format.

Replicat: Runs on the TARGET server. Reads remote trail files and applies changes to the target database using SQL (INSERT, UPDATE, DELETE). One Replicat per target schema/table set.

Flow summary:

Source DB → Extract → Local Trail → Data Pump → Remote Trail → Replicat → Target DB

⭐ Q: What is the difference between Classic Capture and Integrated Capture in GoldenGate?

A:

Classic Capture: GoldenGate reads the archive log files directly using its own log reader. Simple to set up. Works on all Oracle versions. But: Cannot capture compressed tables, encrypted tables (TDE), or certain advanced features. Also needs archive logs to be accessible.

Integrated Capture (IC): GoldenGate works directly with the Oracle LogMiner server (part of Oracle database kernel) to receive changes as Logical Change Records (LCRs). Supports TDE (Transparent Data Encryption), compression, RAC, Exadata, and multitenant PDBs. Available from Oracle 11.2.0.3+. Recommended for all modern Oracle environments.

Configure Integrated Capture:

```
GGSCI> ADD EXTRACT ext1, INTEGRATED TRANLOG, BEGIN NOW
GGSCI> REGISTER EXTRACT ext1 DATABASE
```

Key requirement for Integrated Capture: ENABLE_GOLDENGATE_REPLICATION=TRUE must be set on the source database.

Q: What is a GoldenGate Trail File? What is the difference between a local and remote trail?

A:

A Trail File is a series of compressed binary files where GoldenGate stores extracted change records before they are applied to the target.

Local Trail (source server): Written by the Primary Extract. Located on the source server (e.g., ./dirdat/et000000, et000001...). Maximum size per file = configurable (default 100 MB). When full, new file created automatically with incremented sequence number.

Remote Trail (target server): Written by the Data Pump (which reads local trail and sends over TCP/IP to target). Located on the target server (e.g., ./dirdat/rt000000...). Replicat reads from remote trail.

Key trail management commands:

- ADD RMTTRAIL — register a remote trail on the target
- INFO EXTRACT ext1, SHOWCH — show Extract checkpoints (how far Extract has read)
- GGSCI> VIEW REPORT EXT1 — view the Extract report file for errors
- Logdump utility — read trail file contents for debugging

Q: What is GoldenGate Lag? How do you monitor and reduce it?

A:

Lag is the delay between when a transaction was committed on the SOURCE database and when it was applied on the TARGET database.

Check lag:

```
GGSCI> INFO ALL  -- Shows lag for all processes
GGSCI> LAG REPLICAT rep1  -- Detailed lag for a specific Replicat
```

Types of lag:

- Extract lag = time between transaction commit on source and Extract writing to trail. Should be near 0.
- Replicat lag = time between transaction commit on source and Replicat applying to target. This is the total end-to-end lag.

Common causes and fixes:

- Network latency between source and target → optimize network, use compression in Data Pump.
- Replicat too slow → use Parallel Replicat or Coordinated Replicat mode.
- Large transactions (bulk deletes) → break into smaller batches in the application.
- Target database slow → tune target DB, check for blocking sessions.

Q: Scenario: GoldenGate Replicat has abended (stopped with an error). What do you do?

A:

- ▶ Check the Replicat status: GGSCI> INFO REPLICAT rep1
- ▶ View the error in the report file: GGSCI> VIEW REPORT REP1

Common errors: ORA-00001 (duplicate key), ORA-01403 (no data found for UPDATE), ORA-03114 (lost DB connection)

- ▶ For ORA-00001 (duplicate key — row already exists on target):

```
GGSCI> SEND REPLICAT rep1, SKIPTRANSACTION
-- This skips the failed transaction and moves forward.
-- IMPORTANT: Log what was skipped! Investigate if data is now out of sync.
```

- ▶ For ORA-01403 (UPDATE with no matching row on target):

Add HANDLECOLLISIONS parameter to the Replicat parameter file.

This tells Replicat to INSERT if UPDATE target row doesn't exist.

- ▶ After fixing, restart the Replicat: GGSCI> START REPLICAT rep1
- ▶ Monitor the lag to confirm Replicat is catching up: GGSCI> INFO REPLICAT rep1, DETAIL

✅ Key Points to Remember

✔ GoldenGate flow: Source DB → Extract → Trail → DataPump → Remote Trail → Replicat → Target DB
✔ Integrated Capture (recommended) = uses Oracle's LogMiner. Supports TDE, RAC, compression.
✔ Lag = delay between source commit and target apply. Monitor with: GGSCI> INFO ALL
✔ Abended Replicat: check VIEW REPORT, use SKIPTRANSACTION to skip bad transactions.
✔ Key GoldenGate directories: dirprm (parameter files), dirdat (trail files), dirrpt (reports), dirchk (checkpoints)

SECTION 6: Advanced Performance Tuning

📌 **TOPIC SUMMARY**

Advanced performance tuning goes beyond basic AWR reports. It includes SQL Plan Baselines, Adaptive Query Optimization, histograms, bind variable peeking, cursor sharing, row chaining/migration, library cache tuning, and diagnosing complex wait event patterns.

📖 **DESCRIPTION**

At senior level, a DBA is expected to diagnose performance regressions caused by execution plan changes, tune the optimizer using SQL Plan Baselines or SQL Profiles, understand how the Cost-Based Optimizer (CBO) uses statistics and histograms to make decisions, and troubleshoot advanced wait events like 'library cache: mutex X', 'enq: TX row lock contention', or 'log file sync'. Real-world tuning is 80% SQL tuning and 20% system-level tuning.

💡 **REAL-LIFE EXAMPLE**

Application team reports: 'A query that ran in 0.5 seconds last quarter now takes 45 seconds. Nothing changed in the code!' The DBA investigates: checks DBA_HIST_SQL_PLAN to find the plan changed 3 weeks ago (after a statistics gather), creates a SQL Plan Baseline pinning the old plan, and the query is instantly fast again. Root cause: statistics update caused optimizer to choose a different (worse) index. Solution: SQL Plan Baseline prevents future regressions.

6.1 Advanced SQL Tuning Q&A

⭐ **Q: What is a SQL Plan Baseline (SPB)? How do you use it to fix a performance regression?**

A:

A SQL Plan Baseline is a set of 'approved' execution plans stored for a specific SQL statement. The optimizer can only use plans from the baseline — preventing unexpected plan changes from hurting performance.

Scenario: Query was fast with old plan, new plan is bad. Fix steps:

- Find the SQL_ID of the problem query:

```
SELECT sql_id, elapsed_time, sql_text FROM v\$sql WHERE sql_text LIKE '%WHERE customer_id%';
```

- Load the OLD GOOD plan from AWR history into a baseline:

```
DECLARE
  l_plans PLS_INTEGER;
BEGIN
  l_plans := DBMS_SPM.LOAD_PLANS_FROM_AWR(
    begin_snap  => 123,   -- snap_id when query was fast
    end_snap    => 125,
    sql_id      => 'abc123xyz');
```

```
  DBMS_OUTPUT.PUT_LINE('Plans loaded: ' || l_plans);
END; /
```

- Verify the baseline was created:

```
SELECT sql_handle, plan_name, enabled, accepted FROM dba_sql_plan_baselines
WHERE sql_text LIKE '%customer_id%';
```

- The optimizer will now ONLY use the accepted plan. Query returns to fast performance.
- To evolve (test and accept new plans in future): DBMS_SPM.EVOLVE_SQL_PLAN_BASELINE

Q: What is Bind Variable Peeking? What problem does it cause and what is the fix?

A:

When Oracle first executes a SQL statement with bind variables, it 'peeks' (looks at) the actual bind value at parse time to generate an execution plan. This plan is then cached and reused for ALL future executions — even with very different bind values.

The problem:

- Query: SELECT * FROM orders WHERE region_id = :reg
- First execution: :reg = 'NORTH' (1,000 rows) → Oracle builds a plan using FULL TABLE SCAN.
- Next execution: :reg = 'LONDON' (5 rows) → Oracle REUSES the full scan plan — but an INDEX SCAN would be 1000x faster!

Fix 1 — Adaptive Cursor Sharing (Oracle 11g+):

- Automatically detects when different bind values produce very different cardinalities.
- Creates separate 'child cursors' with different plans for different value distributions.
- Enabled automatically when bind variables are used. Check with: IS_BIND_SENSITIVE and IS_BIND_AWARE in V$SQL.

Fix 2 — Create a SQL Profile that forces a specific plan:

```
  EXEC DBMS_SQLTUNE.ACCEPT_SQL_PROFILE(task_name=>'my_tune_task',
force_match=>TRUE);
```

Fix 3 — Use histograms on the column so the optimizer knows the data distribution:

```
  EXEC
DBMS_STATS.GATHER_TABLE_STATS('SCHEMA','ORDERS',METHOD_OPT=>'FOR
COLUMNS region_id SIZE AUTO');
```

Q: What are Histograms in Oracle? When should you use them?

A:

A Histogram is additional statistics that tell the optimizer how data is distributed within a column — especially for columns with skewed data (some values are very common, others are rare).

Without histogram:

- Oracle assumes all values are equally distributed (uniform distribution).
- If 95% of orders are for 'INDIA' and 5% for 'UK', Oracle might still choose a full table scan for 'UK' query — wrong!

With histogram:

- Oracle knows 'INDIA' = 95% of rows, 'UK' = 0.1% of rows.
- For 'UK' query → uses INDEX (small result). For 'INDIA' query → uses FULL SCAN (large result). Both correct!

Create histogram for a specific column:

```
EXEC DBMS_STATS.GATHER_TABLE_STATS('HR','ORDERS',
  METHOD_OPT => 'FOR COLUMNS country SIZE 254');
-- SIZE 254 = max buckets (frequency histogram)
-- SIZE AUTO = Oracle decides based on data skew
```

Check if histogram exists:

```
SELECT column_name, histogram, num_buckets FROM dba_tab_col_statistics
WHERE table_name='ORDERS' AND column_name='COUNTRY';
```

Q: What is Row Chaining and Row Migration? How do you detect and fix them?

A:

Row Migration: A row was originally small enough to fit in one block. After an UPDATE made it larger, it no longer fits. Oracle moves the entire row to a new block but leaves a 'forwarding address' (pointer) in the original block. When a query accesses this row, Oracle reads TWO blocks instead of one — double the I/O.

Row Chaining: A row is so large (wide table with many VARCHAR2 columns) that it spans multiple blocks from the start. Unavoidable in this case — just reduce row size or increase block size.

Detect row migration:

```
ANALYZE TABLE orders COMPUTE STATISTICS;
SELECT table_name, chain_cnt FROM dba_tables WHERE table_name='ORDERS';
-- chain_cnt > 0 means rows are chained or migrated
```

Fix row migration:

- ▶ Move the table to a new segment (rebuilds all rows compactly): `ALTER TABLE orders MOVE;`
- ▶ Rebuild all indexes on the table (MOVE makes indexes unusable): `ALTER INDEX idx_orders_cust REBUILD;`
- ▶ Prevent future migration: set PCTFREE higher (e.g., 20–30%) if rows are frequently updated: `ALTER TABLE orders PCTFREE 20;`

PCTFREE = percentage of each block kept free for future row growth.

Q: What is the Shared Pool? What causes ORA-04031 and how do you fix it?

A:

The Shared Pool is the SGA component that caches: SQL statements and their execution plans (Library Cache), data dictionary information (Dictionary Cache), and PL/SQL code.

ORA-04031: 'unable to allocate X bytes of shared memory (shared pool)'

- Means: The Shared Pool is full and cannot fit a new SQL statement or PL/SQL object.

Root causes:

- Too many HARD PARSES — application not using bind variables, creating thousands of unique SQL statements that each take space.
- Shared Pool is too small for the workload.
- Memory fragmentation — many small unusable gaps in the pool.

Fix steps:

▶ Short-term: Flush the shared pool (removes cached SQL — performance hit for a few minutes): ALTER SYSTEM FLUSH SHARED_POOL;

▶ Increase Shared Pool size: ALTER SYSTEM SET shared_pool_size=2G SCOPE=BOTH;

▶ Fix the root cause — mandate bind variables in application code.

▶ Set CURSOR_SHARING=FORCE as a temporary measure (Oracle replaces literals with binds automatically): ALTER SYSTEM SET cursor_sharing=FORCE;

✅ Key Points to Remember

✔ SQL Plan Baseline = 'approved plans list' for a SQL. Prevents unexpected plan regressions.
✔ Bind variable peeking problem → fixed by Adaptive Cursor Sharing (11g+) or histograms.
✔ Histogram = tells optimizer about skewed data distribution in a column.
✔ Row Migration → fix with ALTER TABLE MOVE + rebuild indexes + increase PCTFREE.
✔ ORA-04031 = Shared Pool full. Causes: no bind variables, pool too small. Fix: FLUSH SHARED_POOL + increase size.
✔ Key views: DBA_HIST_SQL_PLAN, DBA_SQL_PLAN_BASELINES, DBA_TAB_COL_STATISTICS

SECTION 7: Table and Index Partitioning

📌 **TOPIC SUMMARY**

Partitioning divides a large table or index into smaller, manageable pieces called partitions — each stored separately but appearing as one logical object. This dramatically improves query performance on large tables and simplifies data management (archiving old data, loading new data).

📖 **DESCRIPTION**

Oracle supports several partitioning strategies. Range partitioning is most common — divides data by a range of values like dates (Jan 2024, Feb 2024...). List partitioning groups specific values (NORTH, SOUTH, EAST, WEST). Hash partitioning distributes data evenly across partitions using a hash function — good for load balancing. Interval partitioning automatically creates new partitions as data arrives. Composite partitioning combines strategies (e.g., Range-Hash). Partition Pruning is the key performance benefit — Oracle reads only relevant partitions instead of the entire table.

💡 **REAL-LIFE EXAMPLE**

You have an ORDERS table with 5 billion rows, partitioned by month (range on ORDER_DATE). A query asks for 'all orders from March 2026'. Without partitioning, Oracle scans all 5 billion rows. With monthly range partitioning, Oracle reads ONLY the March 2026 partition (say, 50 million rows). That is 100x less data to scan — query goes from 10 minutes to 6 seconds. This is called Partition Pruning.

7.1 Partitioning Q&A

Q: What are the different types of partitioning in Oracle? Which is most common?

A:

Range Partitioning: Divides rows based on a range of column values. Most common — used on DATE columns in billing, orders, logs. Example: Jan partition, Feb partition...

List Partitioning: Divides rows based on specific discrete values. Example: REGION IN ('NORTH','SOUTH'), STATUS IN ('ACTIVE','INACTIVE').

Hash Partitioning: Distributes rows evenly across N partitions using a hash function. Best when no natural range exists and you want even I/O distribution. Example: hash on CUSTOMER_ID into 16 partitions.

Interval Partitioning: Extension of Range — Oracle AUTOMATICALLY creates new partitions as data arrives. No DBA intervention needed for new time periods. Best practice for any time-series data.

Composite Partitioning: Two-level partitioning: Range-Hash, Range-List, Range-Range. Example: Range by month (top level), Hash by region (sub-partitions). Very powerful for large data warehouses.

Most common in production:

Range on a DATE column (monthly or daily) for transactional tables + Interval partitioning for automatic management.

⭐ Q: What is Partition Pruning? How does it work?

A:

Partition Pruning is the ability of the Oracle optimizer to SKIP reading irrelevant partitions entirely when executing a query.

Example: ORDERS table partitioned by RANGE on ORDER_DATE (monthly):

```
SELECT * FROM orders WHERE order_date BETWEEN '01-MAR-2026' AND '31-MAR-2026';
-- Oracle reads ONLY the March 2026 partition.
-- All other 59 monthly partitions are not touched.
```

Check if partition pruning is happening — look for 'PARTITION RANGE SINGLE' or 'PARTITION RANGE ITERATOR' in explain plan:

```
EXPLAIN PLAN FOR SELECT * FROM orders WHERE order_date = TRUNC(SYSDATE,'MM');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Common mistake that PREVENTS partition pruning:

```
SELECT * FROM orders WHERE TRUNC(order_date,'MM') = '01-MAR-2026';
-- Applying a function to the partition key column disables pruning!
-- Always filter on the partition key directly (no functions on it).
```

Q: What is the difference between a Local Partitioned Index and a Global Partitioned Index?

A:

Local Partitioned Index: One-to-one relationship between data partitions and index partitions. Each index partition covers EXACTLY ONE data partition. When a data partition is dropped or truncated, only its index partition is affected. Best for OLTP — easier maintenance, partition independence.

Global Partitioned Index: Index is partitioned differently from the table. One global index partition can cover multiple table partitions (or the whole table). More flexible but more maintenance — dropping a table partition marks the global index UNUSABLE. Must be rebuilt or maintained with UPDATE INDEXES clause.

Rule of thumb:

- Use LOCAL indexes when: you often drop old partitions (data archiving). Local index partition drops with the table partition — no rebuild.
- Use GLOBAL indexes when: queries access data across many partitions and the query column is not the partition key.

Maintain global index when dropping a partition:

```
ALTER TABLE orders DROP PARTITION orders_jan_2024 UPDATE GLOBAL INDEXES;
```

Q: How do you add a new partition to an existing table and move data to it?

A:

For Range/List partitioned table — add a new partition:

```
ALTER TABLE orders ADD PARTITION orders_apr_2026
  VALUES LESS THAN (TO_DATE('01-MAY-2026','DD-MON-YYYY'));
```

For Interval partitioned table — no need! Oracle creates it automatically when data arrives.

Move data between partitions (Partition Exchange):

- Create a regular table with the same structure:

```
CREATE TABLE orders_apr_staging AS SELECT * FROM orders WHERE 1=0;
```

- Load data into the staging table (very fast — no index maintenance):

```
INSERT /*+ APPEND */ INTO orders_apr_staging SELECT * FROM old_system WHERE month='APR';
```

- Exchange the staging table with the partition (atomic operation — no row movement!):

```
ALTER TABLE orders EXCHANGE PARTITION orders_apr_2026
  WITH TABLE orders_apr_staging INCLUDING INDEXES WITHOUT VALIDATION;
```

Partition Exchange is the fastest way to load large amounts of data — it just swaps metadata, no actual row copying. Used in data warehouses for bulk ETL loads.

✅ Key Points to Remember

✔ Range on DATE = most common. Interval = auto-creates partitions. Hash = even distribution.
✔ Partition Pruning = Oracle reads only relevant partitions. Never apply functions to the partition key in WHERE clause!
✔ Local index = one partition per table partition (easy to maintain). Global index = spans multiple partitions (more flexible).
✔ Partition Exchange = fastest bulk load method — just swaps pointers, no row movement.
✔ Always use UPDATE GLOBAL INDEXES when dropping partitions — prevents index becoming unusable.

SECTION 8: Database Upgrade & Migration

📌 **TOPIC SUMMARY**

Database upgrades (e.g., Oracle 12c to 19c) are high-risk, high-reward activities. Advanced DBAs must know all upgrade methods, pre-upgrade checks, rollback strategies, post-upgrade validations, and zero-downtime upgrade techniques using Data Guard Rolling Upgrade or GoldenGate.

📖 **DESCRIPTION**

Oracle provides multiple upgrade paths. DBUA (Database Upgrade Assistant) is the GUI tool — automated but less flexible. Manual upgrade using SQL scripts gives full control. AutoUpgrade is Oracle's newest recommended tool (from 19c) — it handles pre-checks, upgrade, and post-upgrade automatically for one or many databases. For zero-downtime upgrades, Data Guard Rolling Upgrade is the gold standard: upgrade standby first, switch roles, upgrade old primary as new standby. GoldenGate migration is used when you want to change platforms (OS, hardware, storage) simultaneously with the upgrade.

💡 **REAL-LIFE EXAMPLE**

A company runs Oracle 12.1 on AIX servers. They want to move to Oracle 19c on Linux with zero downtime. The DBA uses GoldenGate: sets up replication from AIX 12.1 to Linux 19c. While replication syncs, they test the new environment. On cutover weekend, they wait for lag to reach 0 seconds, switch the application connection string, and stop the old server. Total application downtime: 2 minutes. This is the power of GoldenGate-based migration.

8.1 Upgrade & Migration Q&A

Q: What is Oracle AutoUpgrade? How is it different from DBUA?

A:

AutoUpgrade (introduced in Oracle 18c, recommended from 19c) is a command-line tool that fully automates the database upgrade process — pre-checks, upgrade, post-upgrade, and validation.

Key advantages of AutoUpgrade over DBUA:

- Upgrade MANY databases in parallel from one command.
- Has a GUARANTEE mode: automatically takes a Guaranteed Restore Point before upgrade — so if upgrade fails, it can automatically flashback to pre-upgrade state.
- Runs pre-checks and generates a detailed report of issues before touching the database.
- Unattended — can be scheduled, runs overnight.

AutoUpgrade workflow:

▶ Run in analyze mode (pre-checks only, no changes):

```
java -jar autoupgrade.jar -config config.cfg -mode analyze
```

▶ Review the pre-check report. Fix any issues.

▶ Run the actual upgrade:

```
java -jar autoupgrade.jar -config config.cfg -mode upgrade
```

- Monitor progress: GGSCI (no — SQL*Plus) job status command or the generated log file.
- Post-upgrade: run utlrp.sql to compile invalid objects, update statistics.

⭐ Q: What is a Data Guard Rolling Upgrade? What are the steps?

A:

Data Guard Rolling Upgrade uses the standby database to achieve near-zero downtime upgrade. You upgrade the standby first while the primary keeps running, then do a fast switchover.

High-level steps:

- Stop redo apply on standby: ALTER DATABASE RECOVER MANAGED STANDBY DATABASE CANCEL;
- Upgrade the standby database to the new Oracle version (using DBUA or AutoUpgrade).
- Start redo apply on standby (transient logical standby converts physical standby temporarily): -- Uses DBMS_ROLLING package in 12c+
- Once standby is fully caught up with primary, perform SWITCHOVER:

```
DGMGRL> SWITCHOVER TO standby_db;
-- Old standby (now upgraded) becomes new primary.
-- Old primary becomes new standby. Application connects to new primary.
```

- Upgrade the old primary (now the standby) to the new version.
- Switchover back if desired (or keep the current setup).

Total application downtime: only the switchover time — typically 30 seconds to 2 minutes.

Q: What pre-upgrade checks must you perform before upgrading Oracle?

A:

Step 1 — Run the Pre-Upgrade Information Tool:

```
sqlplus / as sysdba
@\$NEW_ORACLE_HOME/rdbms/admin/preupgrd.sql
-- Or with AutoUpgrade: java -jar autoupgrade.jar -config db.cfg -mode analyze
```

Step 2 — Check key items from the pre-upgrade report:

- Deprecated/desupported parameters — must be removed from SPFILE before upgrade.
- Invalid objects — run utlrp.sql to compile them first.
- Timezone version — may need to be updated.
- COMPATIBLE parameter — verify target version compatibility.

Step 3 — Take a full RMAN backup of the database before upgrade.

Step 4 — Create a Guaranteed Restore Point:

```
ALTER DATABASE CREATE GUARANTEED RESTORE POINT pre_upgrade_rp
GUARANTEE FLASHBACK DATABASE;
-- This allows flashback to pre-upgrade state if something goes badly wrong.
```

Step 5 — Test the upgrade in a non-production environment FIRST. At least twice.

Step 6 — Get application team sign-off on the downtime window and rollback plan.

✅ Key Points to Remember

✔ AutoUpgrade = Oracle's recommended upgrade tool from 19c. Handles pre-checks, upgrade, and validation automatically.
✔ Data Guard Rolling Upgrade = upgrade standby first, switchover, upgrade old primary. Near-zero downtime.
✔ Always run Pre-Upgrade Information Tool (preupgrd.sql) before any upgrade.
✔ Create Guaranteed Restore Point before upgrade — instant rollback option if upgrade fails.
✔ Post-upgrade: run utlrp.sql to recompile invalid objects + gather optimizer statistics.
✔ GoldenGate migration = upgrade + platform change simultaneously with near-zero downtime.

SECTION 9: Database Security — TDE, VPD & Auditing

📌 **TOPIC SUMMARY**

Advanced Oracle security covers three main areas: TDE (Transparent Data Encryption — encrypts data on disk), VPD (Virtual Private Database — row-level security based on user context), and Unified Auditing (tracking who did what in the database). These are mandatory topics for compliance-heavy industries like banking, healthcare, and government.

📖 **DESCRIPTION**

TDE encrypts Oracle data files so that even if someone steals a hard drive, they cannot read the data without the Oracle Wallet (key store). It is transparent to applications — no application code changes needed. VPD (also called Fine-Grained Access Control — FGAC) automatically adds a WHERE clause to every query based on the user's identity — so employees only see their department's data without the application needing to implement this. Unified Auditing (Oracle 12c+) replaces the old auditing mechanism with a single, tamper-resistant audit trail stored in the AUDSYS schema.

💡 **REAL-LIFE EXAMPLE**

A bank stores customer account data. TDE ensures that if a backup tape is stolen, the thief cannot read the data files. VPD ensures that a branch manager in Mumbai automatically sees only Mumbai branch accounts even if they run SELECT * FROM accounts — Oracle silently adds 'WHERE branch_id = 101' to their query. Auditing records every time someone queries the accounts table, modifies data, or logs in — critical for regulatory compliance (RBI, GDPR, SOX).

9.1 TDE, VPD & Auditing Q&A

Q: What is TDE (Transparent Data Encryption)? How does it work?

A:

TDE encrypts data at the storage level — data files on disk are encrypted, so even if someone copies the .dbf files, they cannot read the data without the Oracle Wallet.

How TDE works:

- ▶ Oracle Wallet stores the master encryption key. Must be opened before the database can access encrypted data.
- ▶ When data is WRITTEN to disk: Oracle encrypts it automatically before writing.
- ▶ When data is READ from disk: Oracle decrypts it automatically before putting it in the buffer cache.
- ▶ Application sees normal data — completely transparent.

TDE can encrypt at two levels:

- COLUMN-level: Only specific sensitive columns (e.g., credit card number, SSN) are encrypted.
- TABLESPACE-level: Entire tablespace is encrypted — all tables in it are automatically encrypted.

Set up TDE (tablespace encryption):

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE '/etc/oracle/wallet' IDENTIFIED BY WalletPass123;
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY WalletPass123;
ADMINISTER KEY MANAGEMENT SET KEY IDENTIFIED BY WalletPass123 WITH BACKUP;
-- Now create encrypted tablespace:
CREATE TABLESPACE secure_data DATAFILE '...' ENCRYPTION USING AES256 DEFAULT STORAGE (ENCRYPT);
```

⭐ Q: What is VPD (Virtual Private Database)? How is it different from normal access control?

A:

VPD (Virtual Private Database) — also called Fine-Grained Access Control (FGAC) — automatically adds a security predicate (WHERE clause) to every SQL statement based on the application context of the user.

Normal access control:

- You GRANT SELECT ON employees TO hr_manager — they can see ALL employee rows.
- To restrict access, the application must add its own WHERE clause — but what if someone uses SQL*Plus directly?

VPD:

- Oracle automatically adds WHERE dept_id = 10 (for a manager in dept 10) to EVERY query on that table.
- This happens at the database engine level — impossible to bypass, even with SQL*Plus.

Create a VPD policy:

```
-- Step 1: Create a policy function that returns the predicate:
CREATE OR REPLACE FUNCTION emp_security(schema_name IN VARCHAR2, table_name IN VARCHAR2)
RETURN VARCHAR2 AS
BEGIN
  RETURN 'dept_id = SYS_CONTEXT("USERENV","CLIENT_INFO")';
END; /
-- Step 2: Add VPD policy to the table:
DBMS_RLS.ADD_POLICY(object_schema=>'HR', object_name=>'EMPLOYEES',
  policy_name=>'EMP_POLICY', function_schema=>'HR',
  policy_function=>'EMP_SECURITY');
```

Q: What is Unified Auditing in Oracle 12c+? How is it different from traditional auditing?

A:

Traditional Auditing (pre-12c): Multiple audit trails in different locations — SYS.AUD$ table (for standard audit), OS audit files, Fine-Grained Audit records. Difficult to manage, could be modified by DBAs.

Unified Auditing (12c+): ALL audit records go to ONE place: the AUDSYS schema in a secure audit trail table. Captures standard audit, FGA audit, RMAN audit, Data Pump audit, and SQL*Plus actions in one unified trail. DBAs CANNOT modify it (tamper-resistant).

Create a Unified Audit Policy:

```
-- Audit all logins and failures:
CREATE AUDIT POLICY login_policy
  ACTIONS LOGON, LOGOFF
  WHEN 'SYS_CONTEXT("USERENV","SESSION_USER") != "SYS"'
  EVALUATE PER SESSION;
AUDIT POLICY login_policy;

-- Audit SELECT on sensitive table by specific user:
CREATE AUDIT POLICY salary_access
  ACTIONS SELECT ON hr.employees
  WHEN 'SYS_CONTEXT("USERENV","SESSION_USER") != "HR"'
  EVALUATE PER ACCESS;
AUDIT POLICY salary_access;
```

Query audit records:

```
SELECT event_timestamp, db_username, action_name, object_name,
client_program_name
FROM unified_audit_trail
WHERE action_name = 'SELECT' AND object_name = 'EMPLOYEES'
ORDER BY event_timestamp DESC;
```

✅ Key Points to Remember

✔ TDE encrypts data on disk — transparent to applications. Uses Oracle Wallet for key management.
✔ TDE levels: COLUMN (specific columns) or TABLESPACE (entire tablespace encrypted).
✔ VPD automatically adds WHERE clause based on user context — database-level row security.
✔ Unified Auditing: single tamper-resistant audit trail. All audit types in one location (AUDSYS).
✔ Key views: V$ENCRYPTION_WALLET, DBA_ENCRYPTED_COLUMNS, UNIFIED_AUDIT_TRAIL, DBA_AUDIT_POLICIES

SECTION 10: Advanced Troubleshooting Scenarios

📌 **TOPIC SUMMARY**

Advanced troubleshooting covers scenarios beyond basic errors — corrupt data blocks, ORA-600/ORA-7445 internal errors, Data Guard apply failures, RAC node evictions, GoldenGate replication failures, and database performance emergencies. These require deep diagnostic skills and systematic approach.

📖 **DESCRIPTION**

At the senior DBA level, the most complex scenarios involve: data corruption (block-level corruption detected by RMAN or DBVERIFY), Oracle internal errors (ORA-600 — Oracle kernel bug, ORA-7445 — process crash with core dump), Data Guard falling behind or apply gaps, performance problems that survive basic AWR analysis, and storage failures. The key skill is systematic diagnosis using Oracle's ADR (Automatic Diagnostic Repository), trace files, and correlated evidence from multiple views and logs.

💡 **REAL-LIFE EXAMPLE**

At 2 AM, the on-call DBA gets a page: 'Application is throwing ORA-01578: ORACLE data block corrupted'. This is a block corruption — actual data on disk is unreadable. The DBA uses RMAN to restore just the corrupted data block from backup (block media recovery) while the rest of the database stays fully online. Users are back in 5 minutes. No data loss. This is real advanced DBA work — the kind that earns promotions and respect.

10.1 Advanced Troubleshooting Q&A

Q: What is ORA-600 and ORA-7445? What do you do when you see them?

A:

ORA-600: Oracle internal error — an unexpected condition inside the Oracle kernel code. Often a bug in Oracle software. Arguments (in brackets after ORA-600) identify the specific internal error code to search on Oracle Support.

ORA-7445: Oracle process crash — a signal was sent to an Oracle background process causing it to dump a core file. More severe than ORA-600. Indicates memory corruption or OS-level issue.

What to do when you see ORA-600 or ORA-7445:

- Check the alert log for the full error details including the arguments: tail -200 $ORACLE_BASE/diag/rdbms/orcl/ORCL/trace/alert_ORCL.log
- Find the associated trace file:

```
SELECT value FROM v\$diag_info WHERE name = 'Default Trace File';
-- The trace file has the full stack trace and context.
```

- Use the ORA-600/ORA-7445 Lookup Tool on Oracle Support (support.oracle.com) — enter the error arguments and Oracle version to find known bugs and patches.
- If the error is causing instability, apply the patch identified by Oracle Support.

▶ If no patch is available, open an SR (Service Request) with Oracle Support — provide the trace file.

⭐ Q: What is a Corrupt Data Block? How do you find and fix corruption?

A:

A corrupt data block is a database block (usually 8 KB) whose content does not match Oracle's expected format — the checksum is wrong or internal pointers are inconsistent. Caused by hardware failure (disk, memory, I/O), OS bugs, or storage issues.

Step 1 — Detect corruption (RMAN validates all blocks):

```
rman target /
BACKUP VALIDATE DATABASE;  -- checks all blocks, does NOT create backup
-- After running, check for corrupt blocks:
SELECT * FROM v\$database_block_corruption;
```

Step 2 — Also check using DBVERIFY (OS utility):

```
dbv file=/u02/oradata/ORCL/users01.dbf blocksize=8192
```

Step 3A — Fix using RMAN Block Media Recovery (fastest — database stays OPEN):

```
rman target /
BLOCKRECOVER CORRUPTION LIST;
-- Restores ONLY the corrupt blocks from backup. No downtime. DB stays open.
```

Step 3B — If the corrupt block is not in backup (too recent corruption):

- If it is a table block: export rows, drop and recreate the table, re-import (data in corrupt block may be lost).
- If it is an index block: just rebuild the index (data is in table, not index).

Q: Scenario: Data Guard apply has stopped. The MRP process is not running. How do you diagnose and fix it?

A:

▶ Check standby status:

```
SELECT process, status, thread#, sequence#, blocks_done FROM v\$managed_standby;
-- If MRP0 is missing, apply has stopped.
```

▶ Check the standby alert log for errors:

```
tail -100 $ORACLE_BASE/diag/rdbms/orclstby/ORCLSTBY/trace/alert_ORCLSTBY.log
-- Look for ORA errors that caused MRP to stop.
```

▶ Common causes:

- ORA-00600 in standby during apply — block corruption on standby. Fix: restore the corrupt datafile from primary.

- Missing archive log (gap) — FAL could not fetch the log. Fix: copy the archive log manually.
- ORA-16766 (apply stopped due to redo gap) — network outage caused missed logs.

▶ Restart MRP manually:

```
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE DISCONNECT FROM SESSION;
```

▶ Verify apply resumes:

```
SELECT process, status, sequence# FROM v\$managed_standby;
-- Should show MRP0 with status APPLYING_LOG
```

▶ Check that the standby is catching up to primary:

```
SELECT name, value FROM v\$dataguard_stats WHERE name LIKE '%lag%';
```

Q: How do you diagnose and resolve ORA-04031 (shared pool out of memory) during peak hours?

A:

▶ Confirm the error and get details:

```
SELECT to_char(originating_timestamp,'DD-MON-YYYY HH24:MI:SS'), message_text
FROM v\$diag_alert_ext WHERE message_text LIKE '%ORA-04031%'
ORDER BY originating_timestamp DESC FETCH FIRST 10 ROWS ONLY;
```

▶ Check current shared pool usage:

```
SELECT pool, name, bytes/1024/1024 AS mb FROM v\$sgastat
WHERE pool = 'shared pool' ORDER BY bytes DESC FETCH FIRST 10 ROWS ONLY;
```

▶ Find the source of hard parsing (non-bind-variable SQL):

```
SELECT COUNT(*) as unique_sql_count, SUM(executions) as total_execs
FROM v\$sql WHERE executions = 1 AND parsing_user_id != 0;
-- If unique_sql_count is huge = application is NOT using bind variables.
```

▶ Immediate fix — flush shared pool (causes brief performance hit):

```
ALTER SYSTEM FLUSH SHARED_POOL;
```

▶ Temporary mitigation — force cursor sharing:

```
ALTER SYSTEM SET cursor_sharing = FORCE SCOPE=BOTH;
```

▶ Increase shared pool size (permanent fix if workload has genuinely grown):

```
ALTER SYSTEM SET shared_pool_size = 3G SCOPE=BOTH;
```

▶ Long-term root cause fix: Work with developers to use bind variables in all SQL.

✅ Key Points to Remember

✔ ORA-600 = Oracle internal bug. ORA-7445 = process crash. Always check trace file + Oracle Support.

✔ Block corruption: detect with RMAN VALIDATE, fix with RMAN BLOCKRECOVER (database stays open).

✔ Data Guard apply stopped: check V$MANAGED_STANDBY, alert log, restart MRP manually.
✔ ORA-04031: Shared Pool full due to hard parsing. Fix: FLUSH SHARED_POOL, CURSOR_SHARING=FORCE, bind variables.
✔ ADR (Automatic Diagnostic Repository) at $ORACLE_BASE/diag/ — centralized location for ALL Oracle diagnostic files.

QUICK REVISION — Advanced Topics Cheat Sheet

Top Advanced Interview Questions You MUST Know Cold

Multitenant

Q: What is the difference between CDB and PDB?

A:

CDB = container with shared infrastructure. PDB = isolated pluggable database inside CDB. One CDB → many PDBs. Connect to PDB using ALTER SESSION SET CONTAINER or TNS service name.

Q: How do you clone a PDB quickly for development?

A:

Use CREATE PLUGGABLE DATABASE dev_pdb FROM prod_pdb. For fastest clone on ACFS storage: add SNAPSHOT COPY — creates PDB in seconds by sharing data blocks with source.

ASM

Q: What are the 3 ASM redundancy levels?

A:

EXTERNAL (no ASM mirroring — rely on hardware RAID). NORMAL (2-way mirroring — needs 2 failure groups). HIGH (3-way mirroring — needs 3 failure groups). Normal is most common in production.

Q: What is ASM rebalancing and how do you control speed?

A:

ASM automatically redistributes data when disks are added/removed. Control speed with ASM_POWER_LIMIT (0=stop, 1=slowest, 1024=fastest). Monitor: SELECT * FROM v$asm_operation;

Data Guard Advanced

Q: What is the ONE command to do a Data Guard switchover using Broker?

A:

DGMGRL> SWITCHOVER TO standby_db; — Broker handles everything. Without Broker, you need 8+ manual SQL commands on both databases in exact order.

Q: What is Fast-Start Failover?

A:

FSFO uses an Observer process (third server) to automatically trigger failover when primary fails — no DBA intervention. Enable: DGMGRL> ENABLE FAST_START FAILOVER; then START OBSERVER on the observer host.

RAC

Q: What is SCAN and why is it better than VIP for application connections?

A:

SCAN = single DNS hostname resolving to 3 IPs. Applications use one connection string regardless of how many nodes exist. VIP = one per node, clients need all VIP addresses hardcoded. SCAN is always preferred.

Q: What causes RAC node eviction?

A:

Missed cluster heartbeats — usually caused by: private interconnect failure, OS hang, extreme memory pressure (OOM killer), or storage timeout. Node that cannot see majority of voting disks evicts itself to protect data.

GoldenGate

Q: What is the GoldenGate data flow?

A:

Source DB → Extract (reads redo logs) → Local Trail Files → Data Pump (sends over network) → Remote Trail Files → Replicat (applies SQL to target) → Target DB.

Q: GoldenGate Replicat abended with ORA-00001 (duplicate key). What do you do?

A:

Run: GGSCI> SEND REPLICAT rep1, SKIPTRANSACTION — skips the failed transaction. Then check if data is out of sync and investigate why the duplicate exists on target.

Performance Tuning Advanced

Q: Query was fast last month, now slow. How do you investigate?

A:

Step 1: Get SQL_ID from V$SQL or AWR.Step 2: Check if execution plan changed — compare DBA_HIST_SQL_PLAN old vs current.Step 3: Check statistics were updated recently — DBA_TAB_STATISTICS.Step 4: If plan regressed after stats update, load old plan into SQL Plan Baseline.Step 5: DBMS_SPM.LOAD_PLANS_FROM_AWR to pin the good old plan.

Q: What is Partition Pruning? What can break it?

A:

Partition Pruning = Oracle reads only relevant partitions by examining the WHERE clause. What breaks it: applying any FUNCTION to the partition key column in WHERE clause (e.g., TRUNC(order_date) instead of filtering order_date directly).

Security

Q: How does TDE protect data? Does it impact performance?

A:

TDE encrypts data files on disk. Even if disk is stolen, data is unreadable. It is transparent to apps (no code changes). Performance impact: 3–5% CPU overhead for encryption/decryption — acceptable for most production systems.

Q: What is VPD and how is it different from GRANT/REVOKE?

A:

GRANT gives all-or-nothing table access. VPD automatically adds a WHERE clause based on user context — so each user sees only their allowed rows, even with SELECT * queries. Impossible to bypass — applied at the Oracle kernel level.

Troubleshooting

Q: ORA-01578: data block corrupted. What is the fastest fix?

A:

Step 1: Check which blocks are corrupt: SELECT * FROM v$database_block_corruption;Step 2: Use RMAN Block Media Recovery — restores ONLY the corrupt blocks from backup while DB stays fully open and online:RMAN> BLOCKRECOVER CORRUPTION LIST;Step 3: After fix, investigate root cause (hardware, I/O subsystem).

Q: Data Guard MRP process stopped applying redo. How do you restart it?

A:

Step 1: Check: SELECT process, status FROM v$managed_standby;Step 2: Check alert log on standby for the error that stopped MRP.Step 3: Fix the root cause (copy missing archive log, resolve gap, fix corrupt block).Step 4: Restart: ALTER DATABASE RECOVER MANAGED STANDBY DATABASE DISCONNECT FROM SESSION;Step 5: Verify: SELECT process, status, sequence# FROM v$managed_standby;

You are now ready for a Senior Oracle DBA interview!

The topics in this guide — Multitenant, ASM, Data Guard Broker, RAC internals, GoldenGate, SQL Plan Baselines, Partitioning, TDE, VPD, and advanced troubleshooting — are exactly what separates a mid-level DBA from a senior DBA.

- Practice explaining every answer OUT LOUD — as if you are in the interview.
- For each topic, write down the 3-step summary on a sticky note. Stick it on your monitor.
- When you answer, always use the structure: What it is → How it works → Real example → When you would use it.
- Mention tools by name: DGMGRL, AutoUpgrade, ASMCMD, GGSCI — this signals real experience.
- End every scenario answer with a prevention tip — this shows production mindset.

Keep learning. Keep practising. The best DBAs never stop studying. 🚀